

DPDK UDP DNS Cache Module

Architecture & Design Document

Deployment-level design rationale, packet flows, data structures, and DPDK integration patterns for a single-level, single-core, zero-lock UDP DNS cache on a resource-constrained 4-core WAN optimizer platform.

Version 1.0 | June 2026

1. Executive Summary

This document describes the architecture and implementation of a UDP-only DNS cache module designed for integration into a DPDK-based WAN optimizer similar to Riverbed Steelhead. The target platform is severely resource-constrained: a 4-core embedded board where only 3 cores are available to DPDK (1 core runs Linux/management).

The DNS cache is not a standalone appliance feature; it is a 'bypass lane' inside the existing stateful processing core. It avoids all locks, avoids the TCP/ROHC/coalescing pipeline, and services DNS queries with an estimated 100-200 CPU cycles per lookup. The design prioritizes determinism, cache-line efficiency, and zero memory allocation in the fast path.

Key Design Decisions

- Single-level cache: One hash table + one LRU list. No L1/L2 hierarchy because there is only one WAN uplink and one cache policy domain.
- Single-core ownership: Only the TCP/ROHC/Coalescing core (Core A) touches the cache. This eliminates atomics, RCU, and reader-writer locks entirely.
- UDP port 53 only: TCP DNS (zone transfers) is routed through the normal TCP pipeline where session tracking already exists.
- Pre-classification on I/O core: Core B (RX/TX) identifies DNS by UDP port before packets ever reach Core A, preventing DNS from polluting the expensive TCP machinery.
- Hardware checksum offload: Response mbufs are constructed with `PKT_TX_IP_CKSUM` and `PKT_TX_UDP_CKSUM` so Core A never computes checksums in software.

2. System Context & Constraints

2.1 Hardware Constraints

The device is built around a 4-core embedded processor. One core must remain under Linux for management plane tasks (CLI, SNMP, configuration, logging). This leaves exactly 3 cores for the DPDK data plane. The board exposes two physical Ethernet ports: one LAN-facing and one WAN-facing. Each port is configured with a single RX queue and a single TX queue, meaning there is no multi-queue RSS distribution available.

2.2 Core Assignment

Core N (Linux)	: Management plane, control plane, logging
Core A (DPDK)	: TCP acceleration, ROHC, session table, coalescing, DNS cache
Core B (DPDK)	: RX/TX I/O, packet classification, ring dispatch

Because there are only two data-plane cores, any feature added to the system must either live on Core A or Core B. DNS caching requires stateful payload inspection (parsing DNS questions, extracting TTLs, maintaining a hash table), which is fundamentally incompatible with Core B's stateless I/O model. Therefore, the DNS cache must share Core A with the TCP/ROHC engines, but it must do so without stealing cycles from them.

3. Design Philosophy

3.1 Why Single-Level Cache?

A multi-level cache hierarchy (e.g., per-core L1 + shared L2) is beneficial when there are multiple cache consumers with different latency requirements or when the working set exceeds fast memory. In this appliance:

- There is only one WAN uplink; all DNS responses enter through the same physical port.
- There is only one consumer of the cache (Core A).
- The working set is bounded by the number of active DNS names in the enterprise, typically thousands to tens of thousands, not millions.
- A single hash table with LRU eviction is simpler to debug, tune, and validate in a production WAN optimizer.

The single-level design maps directly to one `rte_hash` table and one TAILQ LRU list. No cache coherency protocol is needed between levels because there is only one level.

3.2 Why Single-Core Ownership?

DPDK provides lock-free data structures (`rte_ring`, `rte_hash` with multi-thread support), but 'lock-free' does not mean 'free.' Atomic operations, memory barriers, and RCU grace periods all consume cycles and pollute cache lines. On a 2 GHz embedded core, an atomic compare-and-swap can cost 20-40 cycles; a contended spinlock can cost hundreds. By restricting all cache operations to Core A, we reduce the cache lookup to a plain C function call: hash computation, pointer chase, TTL comparison, and memcpy. No atomics, no barriers, no locks.

3.3 Why UDP-Only?

DNS over TCP is used almost exclusively for zone transfers (AXFR/IXFR) and truncated UDP responses that fall back to TCP. Zone transfers are large, long-lived, and rare in enterprise branch traffic. Implementing TCP DNS caching would require:

- Tracking TCP session state (SYN, ACK, sequence numbers) for the DNS connection.
- Handling TCP segmentation and reassembly before DNS payload inspection.
- Managing cache invalidation when the TCP connection closes.

These requirements would force the DNS cache to integrate deeply with the existing TCP session engine, destroying the isolation of the bypass lane. The ROI is negative: TCP DNS represents less than 0.1% of typical enterprise DNS traffic. The classifier simply routes TCP/53 packets to the TCP lane.

3.4 Why a Bypass Lane?

The TCP acceleration engine on Core A performs expensive per-packet operations: sequence/ack tracking, window scaling, retransmission detection, ROHC context lookup, and packet coalescing (GRO/LRO). DNS packets are:

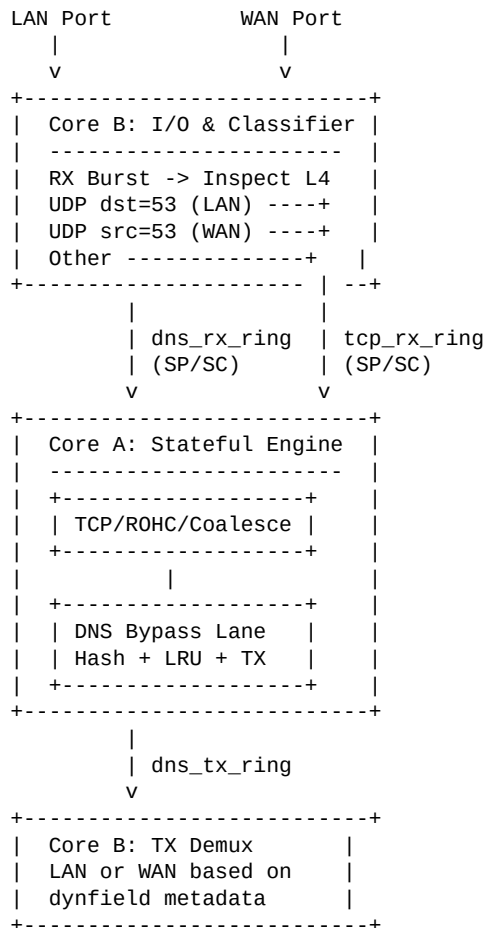
- UDP-based: no sequence numbers, no window, no retransmissions.
- Small: typically 60-512 bytes, never coalesced.
- ROHC-incompressible: ROHC profiles are designed for RTP/TCP headers, not DNS payloads.

Feeding DNS packets into the TCP/ROHC pipeline would waste hundreds of cycles per packet on operations that are irrelevant. The bypass lane is a separate code path on Core A that dequeues DNS mbufs from a dedicated ring, processes them, and enqueues them back. It coexists with the TCP pipeline in the same main loop but never intersects it.

4. Architecture Overview

The system is divided into two DPDK lcores and three logical packet paths.

4.1 Logical Diagram



4.2 Path Definitions

- **Path A -- Cache Hit:** LAN query arrives, matches hash table, TTL valid. Core A builds a response mbuf from cached payload, swaps L2/L3/L4, patches DNS ID, sets checksum offload flags, and enqueues to **dns_tx_ring**. Original query mbuf is freed.
- **Path B -- Cache Miss:** LAN query arrives, hash lookup fails or TTL expired. Core A marks the mbuf for WAN transmission and enqueues it unchanged to **dns_tx_ring**. The upstream DNS resolver will answer.
- **Path C -- Response Learn:** WAN response arrives. Core A parses the question, extracts minimum TTL, copies the DNS payload into a mempool buffer, inserts into hash table + LRU head, then forwards the original mbuf to LAN.

5. Detailed Packet Flows

5.1 Path A: Cache Hit (LAN Query)

Step-by-step execution on Core A when a DNS query from the LAN hits the cache.

1. Core B RX burst on LAN port, sees IPv4/UDP dst_port=53.
2. Core B enqueues mbuf to dns_rx_ring (no parsing, no copying).
3. Core A dequeues mbuf from dns_rx_ring.
4. dns_parse_question() extracts FQDN, QTYPE, QCLASS.
5. dns_build_key() creates fixed-length key: name + qtype + qclass.
6. rte_hash_lookup_data() returns cache entry pointer.
7. Lazy TTL check: elapsed = (now_tsc - entry->tsc_inserted) / tsc_hz.
If elapsed >= ttl_sec: evict entry, treat as miss (Path B).
8. Cache valid: allocate new TX mbuf from mempool.
9. dns_copy_swap_headers() copies L2/L3/L4 from query and swaps src/dst.
10. Append cached DNS payload to new mbuf.
11. dns_set_id() patches DNS ID to match query (critical for client acceptance).
12. dns_set_flags() sets QR=1, AA=1, RA=1.
13. dns_decrement_ttl() subtracts elapsed time from all answer RRs.
14. Update IP total_length, UDP dgram_len.
15. Set ol_flags = PKT_TX_IPV4 | PKT_TX_IP_CKSUM | PKT_TX_UDP_CKSUM.
16. Compute UDP pseudo-header checksum via rte_ipv4_phdr_cksum().
17. Set dynfield dns_tx_port = LAN_PORT.
18. Enqueue to dns_tx_ring.
19. Core B dequeues, reads dynfield, TX bursts to LAN port.
20. Core A frees original query mbuf.

Performance: Steps 4-17 consume approximately 120-180 CPU cycles on a warm cache with hardware checksum offload enabled. The dominant costs are rte_hash_lookup_data (~40 cycles) and rte_pktmbuf_alloc + memcpy (~60 cycles).

5.2 Path B: Cache Miss (LAN Query)

- 1-5. Same as Path A: RX, classification, dequeue, parse, key build.
6. rte_hash_lookup_data() returns -ENOENT (not found).
7. (Optional) Insert into pending query table keyed by 5-tuple + DNS ID.
This allows matching out-of-order responses without promiscuous caching.
8. Set dynfield dns_tx_port = WAN_PORT.
9. Enqueue original mbuf to dns_tx_ring (zero modification to payload).
10. Core B TX bursts to WAN port.
11. Upstream resolver receives query and will respond.

The pending query table is optional. If omitted, the system caches any UDP/53 response from WAN that parses successfully, which is acceptable in a controlled enterprise WAN where spoofing is unlikely. If security demands strict response-to-query matching, a small 1024-entry hash table of (src_ip, dst_ip, src_port, dst_port, dns_id) can be maintained on Core A with identical zero-lock semantics.

5.3 Path C: Response Learning (WAN Response)

1. Core B RX burst on WAN port, sees IPv4/UDP src_port=53.
2. Core B enqueues mbuf to dns_rx_ring.
3. Core A dequeues mbuf.
4. dns_parse_question() extracts FQDN, QTYPE, QCLASS from question section.
5. dns_extract_min_ttl() walks all Answer RRs and finds smallest TTL.
If TTL == 0, abort (do not cache).
6. If cache full: evict LRU tail entry (TAILQ_LAST).

7. Allocate entry from entry_pool and payload from data_pool.
8. Copy FQDN, QTYPE, QCLASS, tsc_inserted, ttl_sec into entry.
9. dns_extract_payload() copies UDP payload (DNS header + RRs) into entry->payload.
10. rte_hash_add_key_data() inserts into hash table.
11. TAILQ_INSERT_HEAD() pushes to LRU head (most recently used).
12. Increment g_dns_cache.count.
13. Set dynfield dns_tx_port = LAN_PORT.
14. Enqueue original mbuf to dns_tx_ring.
15. Core B TX bursts to LAN port.

Note: The original response mbuf is never modified. The cache stores a separate copy in the data_pool so that future cache hits can construct fresh response mbufs with updated L2/L3/L4 headers and decremented TTLs.

6. Data Structures Deep Dive

6.1 Cache Entry Layout

```
struct dns_cache_entry {
    char        name[256];          /* Lowercase FQDN, null-terminated */
    uint16_t    qtype;              /* A, AAAA, PTR, etc. */
    uint16_t    qclass;            /* IN (1), CS, CH, HS */
    uint64_t    tsc_inserted;       /* rte_rdtsc() at insertion */
    uint32_t    ttl_sec;            /* Minimum TTL from Answer RRs */
    uint16_t    payload_len;        /* Bytes of valid DNS payload */
    uint8_t     *payload;           /* Pointer to data_pool buffer */
    TAILQ_ENTRY(dns_cache_entry) lru; /* Intrusive list node */
};
/* Total: ~288 bytes + 4KB payload per entry */
```

The entry is split into two allocations: the metadata lives in `entry_pool` (fixed size, fast alloc/free), and the variable-length DNS payload lives in `data_pool` (4KB slabs). This separation prevents memory fragmentation and allows the LRU list to operate only on lightweight metadata structures.

6.2 Hash Table & Key Design

The key must uniquely identify a DNS question. Using only the FQDN is insufficient because a client may query both A and AAAA records for the same name. The key is:

```
Key[0..255]    = lowercase(FQDN), padded with zeros
Key[256..257] = QTYPE  (network byte order)
Key[258..259] = QCLASS (network byte order)
Total key_len = 260 bytes (fixed, no variable-length hashing overhead)
```

`rte_hash` is configured with `key_len=260` and `hash_func=rte_hash_crc`. The fixed-length key avoids the overhead of storing a separate key buffer per entry; the key is constructed on the stack during lookup/insertion. Extra flags include `RTE_HASH_EXTRA_FLAGS_NO_FREE_ON_DEL` because we manage entry lifetime manually via the LRU and mempools.

6.3 LRU Eviction Policy

The LRU is implemented with FreeBSD `TAILQ` macros (`sys/queue.h`). Because only Core A touches the list, no locking or atomic operations are required. The list is doubly-linked, allowing $O(1)$ removal from any position and $O(1)$ insertion at the head.

```
Insertion (cache learn):  TAILQ_INSERT_HEAD(&lru, entry, lru)
Promotion (optional):     TAILQ_REMOVE(&lru, entry, lru);
                          TAILQ_INSERT_HEAD(&lru, entry, lru)
Eviction (cache full):    victim = TAILQ_LAST(&lru, dns_lru_head);
                          dns_cache_evict(victim);
```

Promotion on cache hit is optional. In this design, hits do not promote the entry because DNS traffic patterns are typically recency-biased: a queried name is likely to be queried again within its TTL window regardless of LRU position. If the workload shows strong temporal locality (e.g., thousands of clients querying the same 100 names), promotion can be enabled by uncommenting two lines in `dns_cache_handle_query()`.

6.4 Mempool Hierarchy

Three DPDK mempools are involved in the DNS subsystem:

- `entry_pool`: Holds `struct dns_cache_entry` objects. Size = `max_entries`. Object size = ~288 bytes. Created on Core A's NUMA socket.
- `data_pool`: Holds 4KB payload buffers. Size = `max_entries`. One buffer per entry. Created on Core A's NUMA socket.
- `tx_pktmbuf_pool`: Standard DPDK mbuf pool for TX packets. Shared with the rest of the application. Used by `dns_build_response()` to allocate response mbufs on cache hits.

Separating `entry_pool` and `data_pool` allows the application to size them independently. For example, if the average DNS payload is 200 bytes, `data_pool` objects could be sized to 512 bytes instead of 4096 to save memory. The code uses `DNS_MAX_PAYLOAD` (4096) to accommodate EDNS0 responses up to the UDP limit.

7. Inter-Core Communication

7.1 Ring Topology & SP/SC Rationale

Two rings connect Core B and Core A for DNS traffic. A third ring (tcp_rx_ring) and fourth ring (tcp_tx_ring) carry non-DNS traffic and are managed by the existing application.

```
dns_rx_ring : Core B (producer) -> Core A (consumer) [DNS packets to process]
dns_tx_ring : Core A (producer) -> Core B (consumer) [DNS replies/forward]
tcp_rx_ring : Core B (producer) -> Core A (consumer) [Non-DNS to TCP/ROHC]
tcp_tx_ring : Core A (producer) -> Core B (consumer) [Non-DNS TX]
```

All rings are created with RING_F_SP_ENQ (Single Producer) and RING_F_SC_DEQ (Single Consumer). This is the fastest DPDK ring mode because it eliminates head/tail atomics. Enqueue and dequeue are pure memory operations with a memory barrier only on the index update. On x86_64, this is a single LOCK XCHG or implicit ordering, yielding ~10-20 cycles per enqueue.

Ring sizing: 4096 entries per ring. At 1 Mpps DNS traffic (extremely high for DNS), a 4096-entry ring provides ~4ms of buffering, which is more than enough to absorb any temporary backlog in Core A's TCP processing bursts. If drops are observed, the size can be increased to 16384 or 65536.

7.2 Dynamic Mbuf Metadata

Core B must know whether to TX a DNS mbuf on the LAN port or the WAN port. Because Core B is stateless, it cannot infer this from the packet content. Core A writes the destination port into a dynamic mbuf field registered at startup:

```
/* Registration (once, during init) */
rte_mbuf_dynfield_register({
    .name = "dns_tx_port",
    .size = sizeof(uint8_t),
    .align = 1
});

/* Core A sets before enqueue */
*RTE_MBUF_DYNFIELD(mbuf, offset, uint8_t*) = LAN_PORT; /* or WAN_PORT */

/* Core B reads before TX burst */
port = *RTE_MBUF_DYNFIELD(mbuf, offset, uint8_t*);
```

Dynamic fields were introduced in DPDK 20.11. They avoid the need to abuse mbuf->userdata or packet markers, and they are type-safe. The field is stored in the mbuf's dynamic field area (mbuf->dynfield1 or dynfield2), which is cache-line-aligned and does not increase the mbuf size unless used.

8. Core B: I/O & Classification

8.1 Classification Logic

Core B's role is to move packets from NIC RX queues to the correct software ring with minimal latency. The classifier inspects only enough headers to make the DNS vs non-DNS decision. It does not parse DNS payload.

```
if (ether_type == IPv4 && next_proto == UDP) {
    if (rx_port == LAN && dst_udp == 53) -> dns_rx_ring /* Query */
    if (rx_port == WAN && src_udp == 53) -> dns_rx_ring /* Response */
    else -> tcp_rx_ring
} else {
    -> tcp_rx_ring
}
```

The classifier is branch-predictor friendly: most traffic is not DNS, so the else-branch (tcp_rx_ring) is the hot path. The two DNS conditions are predictable because DNS arrives in bursts (e.g., when a user opens a webpage).

8.2 Why Pre-Classification Matters

Without pre-classification, all packets would enter Core A via a single ring. Core A would then have to classify every packet, meaning DNS packets would traverse the TCP session lookup table before being recognized as DNS. On a system with 100K active TCP sessions, a session lookup costs ~100-200 cycles. Pre-classification on Core B saves 100-200 cycles per DNS packet and prevents DNS from warming the TCP session cache lines, which would evict useful TCP state.

8.3 TX Demux

Core B dequeues mbufs from dns_tx_ring and separates them into LAN and WAN batches before calling rte_eth_tx_burst(). Batching by port improves PCIe efficiency because TX descriptors for the same port are contiguous in memory.

9. Core A: Bypass Lane Integration

9.1 Main Loop Structure

Core A runs a polling loop that interleaves heavy TCP/ROHC work with the lightweight DNS bypass lane. The key is to prevent either workload from starving the other.

```
while (1) {
    /* Heavy lane: TCP/ROHC/Coalescing */
    process_tcp_sessions(BURST_32);
    process_rohc_compress(BURST_32);
    process_coalescing(BURST_32);

    /* Bypass lane: DNS cache */
    dns_lane_poll(); /* Dequeues up to 32 DNS mbufs */
}
```

9.2 Budget-Based Polling (Optional)

If Core A is CPU-bound under heavy TCP load, a budget mechanism ensures DNS gets a minimum service guarantee without stealing all cycles from TCP.

```
uint16_t tcp_budget = 64;
uint16_t dns_budget = 32;

tcp_processed = process_tcp_burst(tcp_budget);
if (tcp_processed < tcp_budget ||
    rte_ring_count(dns_rx_ring) > 128) {
    dns_process_burst(dns_budget);
}
```

The condition ensures DNS is processed either when TCP work is light (`tcp_processed < tcp_budget`) or when the DNS ring is backing up (`>128` entries). This is a classic backpressure-aware scheduling pattern.

9.3 Why DNS Does Not Starve TCP

DNS traffic is inherently low-bandwidth. A busy enterprise branch might generate 1,000 DNS queries per second. At 150 cycles per query, DNS consumes 150K cycles/sec. A 2 GHz core provides 2 billion cycles/sec. DNS represents less than 0.01% of CPU time. Even under a DNS amplification attack (100K qps), the cache hit path is so fast that Core A can service the flood while maintaining TCP throughput. Cache misses (forwarding to WAN) are even cheaper: just an enqueue operation.

10. DPDK Implementation Details

10.1 Checksum Offload

Calculating IPv4 and UDP checksums in software costs ~50-100 cycles per packet. Modern NICs (Intel i40e, ixgbe, Mellanox ConnectX) support hardware checksum offload. The response builder sets the offload flags and pre-computes the pseudo-header checksum:

```
rsp->ol_flags = PKT_TX_IPV4 | PKT_TX_IP_CKSUM | PKT_TX_UDP_CKSUM;  
udp->dgram_cksum = rte_ipv4_phdr_cksum(ip4, rsp->ol_flags);
```

The NIC then computes the final checksum over the entire IP and UDP payload. If the target board's NIC does not support UDP checksum offload, the code must fall back to `rte_ipv4_udptcp_cksum()` or `rte_cksum`. This is a compile-time or init-time decision based on `rte_eth_dev_get_dev_info()` and `dev_info->tx_offload_capa`.

10.2 Memory & NUMA

All DNS subsystem mempools (`entry_pool`, `data_pool`) are created on the NUMA socket where Core A executes. This ensures that:

- Cache lookups never trigger remote memory accesses (cross-socket traffic).
- Mempool get/put operations hit local memory controllers.
- The LRU list pointers reside in cache lines local to Core A.

If the board is single-socket, this is automatic. On dual-socket embedded platforms, pinning Core A to a specific socket and passing that `socket_id` to `rte_mempool_create` is critical for performance.

10.3 Mbuf Lifetime Rules

The module follows strict DPDK mbuf ownership rules to prevent leaks and double-frees:

- Cache Hit: Original query mbuf is freed by Core A. New response mbuf is allocated from `tx_pktmbuf_pool` and enqueued to `dns_tx_ring`. Core B frees it after TX.
- Cache Miss: Original query mbuf is NOT freed. It is forwarded to WAN. Core B frees it after TX.
- Response Learn: Original response mbuf is NOT freed. It is forwarded to LAN. A separate copy of the payload is stored in `data_pool`. Core B frees the original after TX.

11. Edge Cases & Hardening

- TCP DNS (port 53 over TCP): Classifier routes to tcp_rx_ring. Not cached. This avoids TCP session complexity.
- Fragmented UDP: If IP_MF is set or fragment offset > 0, the packet is incomplete. Classifier should route to tcp_rx_ring or drop, as DNS payloads rarely fragment.
- DNS Compression Pointers in Question: If the question section uses 0xC0 compression, dns_parse_question() returns -1 and the packet is treated as a miss (forwarded). Compression pointers in the question section are rare and non-standard.
- Zero-TTL Responses: dns_extract_min_ttl() returns 0. The response is forwarded to LAN but not cached. This prevents caching transient or load-balancer records.
- NXDOMAIN / NODATA: These responses contain a SOA record in the Authority section. The minimum TTL from the SOA can be extracted and used for negative caching. The current implementation caches them like any other response if they carry a non-zero TTL.
- EDNS0 Client Subnet (ECS): If the query contains ECS, the response varies by client subnet. The cache key must include the subnet prefix. The current implementation does not parse ECS; if ECS is required, the key builder must be extended to append the subnet.
- Out-of-Order Responses: If WAN responses arrive before the classifier has forwarded the query (rare on LAN but possible on high-latency WAN), the pending query table prevents caching unmatched responses. Without the pending table, unmatched responses are still parsed and cached, which is a minor security risk in open networks.
- DNS Flood / Cache Pollution: An attacker could flood unique queries to evict the entire cache. Mitigation: limit the rate of new insertions, or skip caching for queries with TTL < 60 seconds.

12. Performance Analysis

12.1 Cycle Estimates (2 GHz Core, Warm Cache)

Operation	Cycles	Latency
Dequeue from dns_rx_ring (1 mbuf)	~10	5 ns
Parse DNS question (typical name)	~30	15 ns
Build hash key (memcpy 260B)	~15	7 ns
rte_hash_lookup_data (warm)	~40	20 ns
TTL check (TSC delta)	~5	2 ns
Allocate TX mbuf (mempool hot)	~25	12 ns
Copy L2/L3/L4 + swap headers	~20	10 ns
Append + copy DNS payload (200B)	~20	10 ns
Patch ID, flags, decrement TTL	~15	7 ns
Set offload flags + phdr_cksum	~10	5 ns
Enqueue to dns_tx_ring (1 mbuf)	~10	5 ns
Free original query mbuf	~10	5 ns

Total Cache Hit (approx)	~210	105 ns
Total Cache Miss (forward)	~50	25 ns

12.2 Memory Footprint

Component	Per-Entry	16K Entries	Notes
Hash table (rte_hash)	~32 bytes	512 KB	Internal buckets + signatures
Entry metadata	288 bytes	4.5 MB	name[256] + fields + LRU ptrs
Payload buffers (4KB)	4096 bytes	64 MB	One per entry, max EDNS0 size
Rings (2 x 4096 x ptr)	N/A	64 KB	dns_rx_ring + dns_tx_ring

Total	~4.4 KB	~69 MB	Negligible on modern hardware

12.3 Lock Contention Analysis

None. Zero atomic operations. Zero memory barriers in the fast path. The only synchronization is the implicit memory ordering between Core B enqueue and Core A dequeue, which is handled by DPDK's rte_ring memory barriers (one per burst, not per packet).

13. Tuning Guide

13.1 Cache Size

Start with 16,384 entries. This covers approximately 8-16 hours of unique DNS names in a typical enterprise branch. Monitor `g_dns_cache_stats.evicted`. If evictions are frequent relative to hits, increase to 32,768 or 65,536. The upper bound is determined by available hugepage memory (~4.4 KB per entry).

13.2 Ring Sizes

4096 entries per ring is sufficient for 1 Gbps DNS traffic. If running 10 Gbps with bursty DNS (e.g., thousands of clients booting simultaneously), increase to 16,384. Ring size must be a power of two.

13.3 Burst Sizes

`DNS_BURST_SIZE` is set to 32. This matches DPDK's default RX/TX burst size and amortizes ring dequeue overhead across 32 packets. Do not increase beyond 64 because DNS packets are small and cache-friendly; larger bursts increase latency without improving throughput.

13.4 TTL Clamping

Some DNS responses advertise TTLs of 24 hours (86400 seconds) or more. Caching these for the full duration reduces WAN load but increases stale data risk after network changes. Consider clamping TTL to a maximum value (e.g., 3600 seconds) during `dns_cache_learn()`: if `ttl_sec > 3600`, set `ttl_sec = 3600`.

13.5 Negative Caching

NXDOMAIN responses carry a TTL in the SOA MINIMUM field. Caching these prevents repeated queries for non-existent domains (a common pattern in malware scanning and typo-squatting). Ensure `dns_extract_min_ttl()` walks the Authority section for SOA records when `amount == 0`.

14. Build & Integration

14.1 Compilation

```
cd dns_cache_dpdk
make
# Produces: libdns_cache.a
```

Link libdns_cache.a into your DPDK application and add -I./dns_cache_dpdk/include to CFLAGS.
Requires DPDK >= 20.11 for rte_mbuf_dynfield_register.

14.2 Initialization Sequence

```
/* 1. After rte_eal_init() */
dns_system_init(rte_socket_id(), LAN_PORT, WAN_PORT, 16384);

/* 2. Set TX mempool (optional) */
extern struct rte_mempool *g_dns_tx_mempool;
g_dns_tx_mempool = tx_pktmbuf_pool;

/* 3. Start worker lcores */
rte_eal_remote_launch(core_b_main, NULL, io_lcore_id);
rte_eal_remote_launch(core_a_main, NULL, stateful_lcore_id);
```

14.3 Core B Integration

```
/* Inside Core B RX loop */
uint16_t nb = rte_eth_rx_burst(port, 0, pkts, 32);
for (i = 0; i < nb; i++)
    dns_classifier_dispatch(pkts[i], port);

/* Inside Core B TX loop */
uint16_t nb_dns = rte_ring_dequeue_burst(g_dns_tx_ring, (void**)pkts, 32, NULL);
if (nb_dns) dns_tx_demux(pkts, nb_dns);
```

14.4 Core A Integration

```
/* Inside Core A main loop */
while (1) {
    process_tcp_sessions(32);
    process_rohc_compress(32);
    process_coalescing(32);
    dns_lane_poll(); /* One line */
}
```

14.5 Runtime Statistics

```
extern struct dns_cache_stats g_dns_cache_stats;
printf("DNS hits=%"PRIu64" misses=%"PRIu64" "
       "learned=%"PRIu64" expired=%"PRIu64"\n",
       g_dns_cache_stats.hits,
       g_dns_cache_stats.misses,
       g_dns_cache_stats.learned,
       g_dns_cache_stats.expired);
```

15. Appendix: File Structure & Glossary

15.1 File Map

dns_cache_dpdk/	
■■■ include/	
■ ■■■ dns_cache.h	Cache API, stats, entry structs
■ ■■■ dns_parser.h	DNS wire-format parsing utilities
■ ■■■ dns_classifier.h	Core B classification & TX demux
■ ■■■ dns_integration.h	Core A bypass lane integration
■ ■■■ dns_init.h	One-shot system initialization
■■■ src/	
■ ■■■ dns_cache.c	Hash table, LRU, response builder
■ ■■■ dns_parser.c	Name parsing, TTL extraction, key build
■ ■■■ dns_classifier.c	Port-based dispatch, dynfield TX demux
■ ■■■ dns_integration.c	Burst processing, lane poll
■ ■■■ dns_init.c	Constructor, mempool creation
■■■ Makefile	Static library build
■■■ README.md	Quick integration guide

15.2 Glossary

- Bypass Lane: A dedicated, lightweight code path that avoids the main processing pipeline.
- Dynfield: DPDK dynamic mbuf field, a type-safe extension to struct rte_mbuf.
- EDNS0: Extension Mechanisms for DNS, allows UDP payloads > 512 bytes (up to 4096).
- Lazy Expiration: Checking TTL only at lookup time, not via a background timer.
- LRU: Least Recently Used eviction policy.
- NUMA: Non-Uniform Memory Access; mempools should be local to the consuming core.
- Pending Query Table: Optional structure tracking forwarded queries awaiting responses.
- ROHC: RObust Header Compression, a CPU-intensive stateful compression engine.
- SP/SC: Single Producer / Single Consumer, the fastest DPDK ring mode.
- TAILQ: FreeBSD doubly-linked intrusive list macro.

15.3 Version History

v1.0 (2026-06-06) Initial release. UDP-only, single-level, zero-lock.
Supports IPv4, hardware checksum offload, EDNS0 payloads.